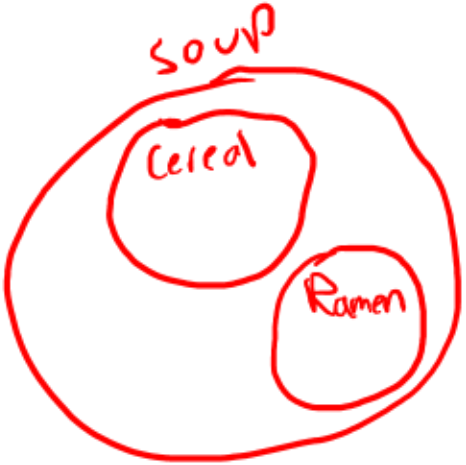
Is cereal a soup?

Is ramen cereal?



CS 340

HTTP

clicker.cs.illinois.edu

Q1

~Code~

~~101~~

340

GO Blue!

M

Updates

1. Attendance updated (let me know if it's not showing up)
2. MP6 graded - Report an error for a regrade request
3. MP 7 is due today
4. MP 8 - aiohttp lite, due next week
5. HW 6 due Wednesday

HTTP

Today's LGs - Start building your understanding of the layers of an internet packet, including HTTP

Be able to describe each of the three layers and what they accomplish
Be able to describe the format of HTTP
Be able to complete MP 8

- Decorators
- Async
- ect...

Agenda

1. Review Clicker

2. HTTP

3. Advanced/Specific Python

Which are True?

A) The internet is a system of computers True

B) Packet~~s~~ ~~are~~ is a continuous stream~~s~~ of bytes F

C) IP addresses specifies which computer a packet needs to go to True

D) TCP is slower but more reliable than UDP True

E) My LAN has a private IP address only I know. F

clicker.cs.illinois.edu

Q2

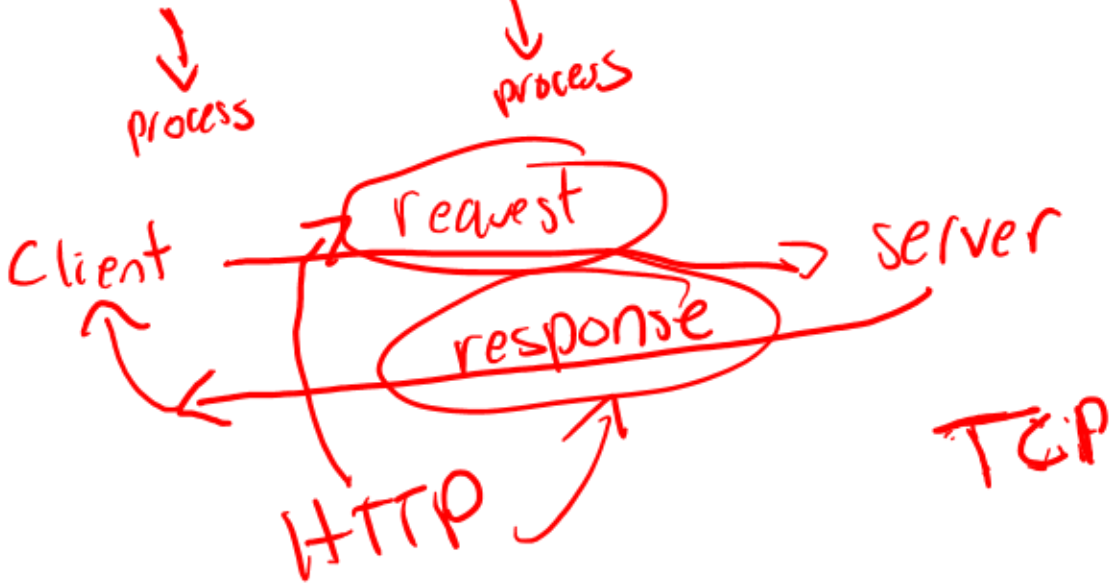
~Code~
340

Layers of a packet

Layer	Protocols	Use
Application	HTTP, SSH, SMTP, etc.	programmer-visible messages
Transport	**TCP, UDP**	reach a specific program
Internet	**IP**, which has two versions: IPv4 and IPv6	reach a specific computer

HTTP

- hyper text transfer protocol
- server and client



HTTP

Request

GET - asks for a resource

POST - sends data to the server

PUT - sends data to the server (overwrites existing)

Response

200 - ok

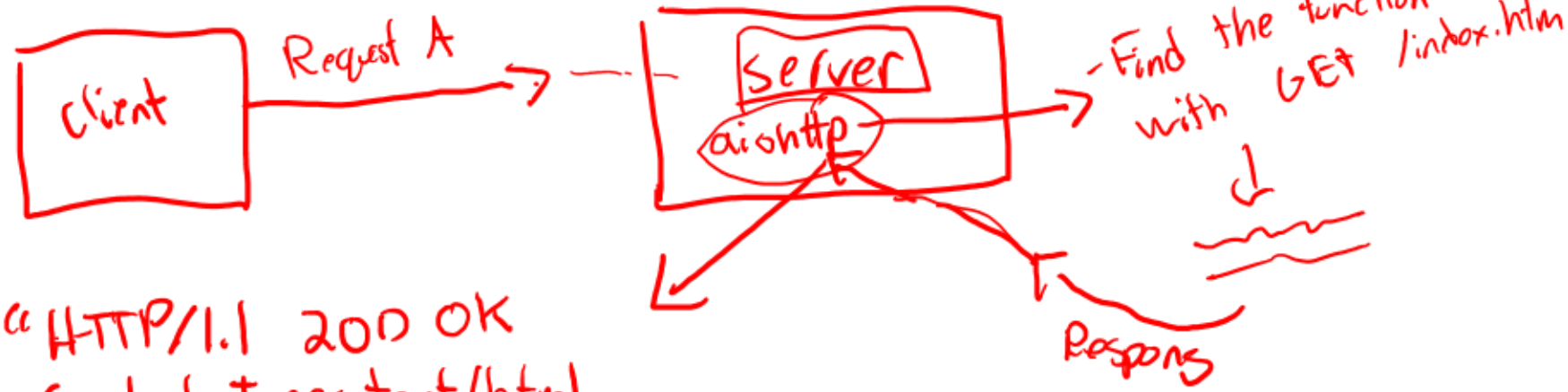
400 - bad request

404 - not found

500 - internal Server Error

HTTP - Format Example 0b01

"GET /index.html HTTP/1.1
Host: example.com" ← Request A



"HTTP/1.1 200 OK
Content-Type: text/html

<html> Hello! </html>"

HTTP - Format Example 0b10

"POST /fav-color HTTP/1.1
Host: example
Content-Type: text/plain

Blue"

"HTTP/1.1 200 OK
Content-Type: text/plain

"Got it!"

What internal numbering system is used to get the HTTP info to the right process?

port

TCP

clicker.cs.illinois.edu

Q3

~Code~
101

Can a client send an HTTP request to a server physically located somewhere else without the IP layer?

clicker.cs.illinois.edu

Q4

~Code~
101

What gets an HTTP request to the correct server function once the packet is at the correct process?

clicker.cs.illinois.edu

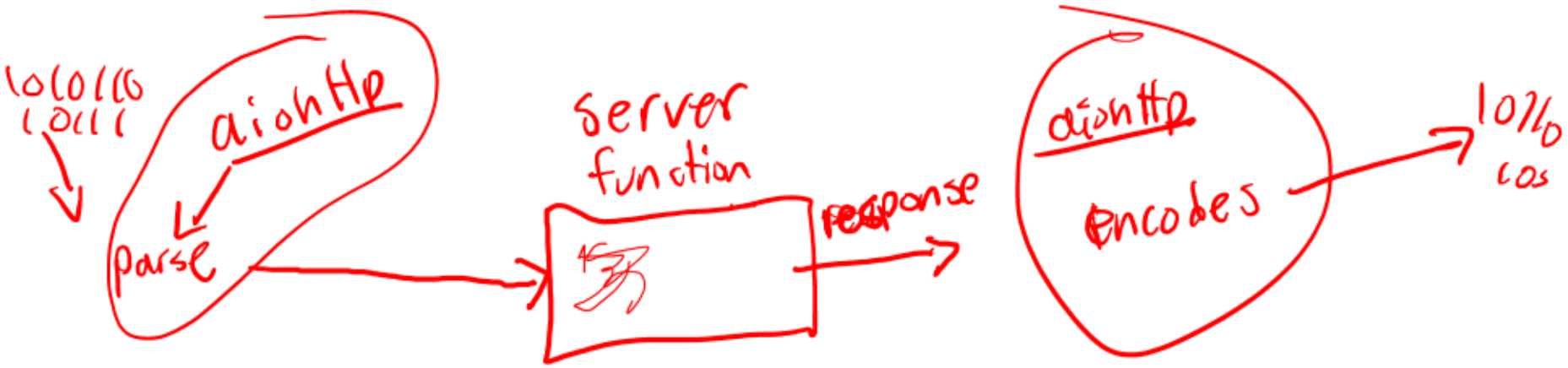
Q5

~Code~
101

the url and request type

Aiohttp - library that parses the bytes of a HTTP request

- send the request body to the correct function
- formats and sends out the response



MP 8 - aiohttp Lite

1. Read in bytes from an incoming packet
2. Interpret the bytes to understand the incoming request
3. Funnel the request body to the corresponding function
    a. The function will call your response constructor to create a response
4. Send that response out

Using your aiohttp

"GET /hint . . . "

```
60  @routes.get("/hint")
61  async def give_game_info(request: web.Request) -> web.Response:
62      return web.Response(text="no hints!")
63
64
65  if __name__ == '__main__':
66      app = web.Application()
67      app.add_routes(routes)
68      web.run_app(app) # this function never returns
```

"HTTP/1.1 200 OK . . . no hints"

Agenda

1. Review Clicker

1. HTTP

1. **Advanced/Specific Python**
    a. **Json**
    b. **Bytes**
    c. **Function decorators**
    d. **Aysnc**

JSON

```
{
  "name": "Jule",
  "favorites": ["CS", "coffee"],
  "info": {
    "year": 2026
  }
}
```

Bytes in Python

```
tmp = "hello"
tmp = tmp.encode("utf-8")
tmp = tmp.decode("utf-8")
```

tmp → "hello"
5
str

→ b'hello'

Are encode() and decode() in-place? Or do they return a copy?

clicker.cs.illinois.edu

Q6

~Code~
101

tmp = tmp.encode('utf-8')

in-place

Bytes in Python Example

main.py

```python
'''Example 17'''
var = "start"
x = [1, 4, 6]
for value in x:
    var += " " + str(value)
print(var)
bytes_version = var.encode("utf-8")
print(bytes_version)
```

```
start 1 4 6
b'start 1 4 6'
```

Classes, what would print?

```
class Book:
    rating : int = 0
    def __init__(self, title: str, length: int):
        self.title = title
        self.length = length

    def add_rating(self, rating: int):
        self.rating = rating

    def should_read(self):
        if self.rating > 5 and self.length > 100:
            return "yes"
        else:
            return "should not read"

bk = Book("from C to shining C++", 400)
print(bk.should_read())
```

clicker.cs.illinois.edu

Q7

~Code~
101

Should not read

Classes + Bytes

```python
class Book:
    rating : int  = 0
    def __init__(self, title: str, length: int):
        self.title = title
        self.length = length

    def __bytes__(self):
        out = self.title + ": " + str(self.length)
        return out.encode('utf-8')

bk = Book("from C to shining C++", 400)
print(bytes(bk))
```

PROBLEMS OUTPUT TERMINAL 3

TERMINAL POR

```
root@3cb3343e38be:/workspaces/mp7# python3 class.py
b'from C to shining C++: 400'
root@3cb3343e38be:/workspaces/mp7#
```

Agenda

1. Review Clicker

1. HTTP

1. **Advanced/Specific Python**
    a. **JSON**
    b. **Bytes**
    c. **Function decorators**
    d. **Aysnc**

Functions - first-class objects

```
def greet():
    print("hi")

f = greet
f()
```

hi

Functions - first-class objects

```
def greet():
    print("hi")
```

```
def run(func):
    func()

run(greet)
```

greet

hi

Functions - first-class objects

```
def make_greeter(name):
    def greet():
        print(f"Hello {name}!")
    return greet

greeter = make_greeter("Jule")
greeter()
```

Hello Jule

greet
name = "Jule"

Greeter

What prints?

```
def add_one(x):
    return x + 1

def run(f, value):
    return f(value)

print(run(add_one, 4))
```

4

5

clicker.cs.illinois.edu

Q8

~Code~
101

What goes in the black box?

```
routes = {}

def register(path, func):
    routes[path] = func

def hello():
    print("Hello!")

register("/hello", [redacted])

routes["/hello"]()
```

Hello!

clicker.cs.illinois.edu

Q9

~Code~
101

hello

Function Decorators

```python
def log(func):
    def wrapper(input_):
        print("called with", str(input_))
        val = func(input_)
        print("finished")
        return val
    return wrapper

@log
def add_num(var):
    return var + 5

print("output:", add_num(5))
```

input

```
called with 5
finished
output: 10
```

add_num = log(add_num)

@log
def add_num:

What prints?

```
def call_twice(func):
    def wrapper(in_):
        func(in_)
        func(in_)
    return wrapper

@call_twice
def print_names(names):
    for i in names:
        print(i)

names_ = ["jule", "luther"]
print_names(names_)
```

print_names = call_twice(print_names)

jule luther
jule luther

clicker.cs.illinois.edu

Q10

~Code~
101

Function Decorators

```
routes = web.RouteTableDef()

@routes.get("/hello")
async def hello(request):
    return web.Response(text="Hello!")
```

hello = routes.get("/hello")(hello)

```python
routes = web.RouteTableDef()

@routes.get("/hello")
async def hello(request):
    return web.Response(text="Hello!")
```

```python
class RouteTableDef:
    def __init__(self):
        self.routes = {}

    def get(self, path):
        def decorator(func):
            self.routes[("GET", path)] = func
            return func
        return decorator
```

/hello

Aiohttp function decorators

```
60    @routes.get("/hint")
61    async def give_game_info(request: web.Request) -> web.Response:
62        return web.Response(text="no hints!")
63
64
65    if __name__ == '__main__':
66        app = web.Application()
67        app.add_routes(routes)
68        web.run_app(app) # this function never returns
```

Agenda

1. Review Clicker

1. HTTP

1. **Advanced/Specific Python**
    a. **Json**
    b. **Bytes**
    c. **Function decorators**
    d. **Aysnc**

Async

fancy function

Python aysnc - let's a program switch between coroutines while one is waiting for something slow (waiting for messages over the internet)

```
async def greet();
    print("hi")
    → await asyncio.sleep(1)

await greet()
```

Async

```python
import asyncio

async def task1():
    print("task1 start")
    await asyncio.sleep(1)
    print("task1 done")

async def task2():
    print("task2 start")
    await asyncio.sleep(2)
    print("task2 done")

async def main():
    await asyncio.gather(task1(), task2())

asyncio.run(main())
```

```
task1 start
task2 start
task1 done
task2 done
```

Async

Idea - async let’s the program switch between coroutines while one is waiting for something slow (waiting for messages over the internet)

- Use the await keyword to indicate the program can switch if its waiting.
- await can be used to call another async function

Async Aiohttp

```python
from aiohttp import web

routes = web.RouteTableDef()

@routes.post("/echo")
async def echo(request):
    data = await request.text()
    return web.Response(text=data)

app = web.Application()
app.add_routes(routes)

web.run_app(app)
```

Async aiohttp lite

Why do you think we use HTTP with TCP and not UDP?

"GET /hello HTTP/1.1"

"_____"

clicker.cs.illinois.edu

Q11

~Code~
101

Where should the await go?

```
@routes.post("/data")
async def handler(request):
    data = request.text()
    return web.Response(text=data)
```

clicker.cs.illinois.edu

Q12

~Code~
101